Comparing Forecasters for Serverless Invocation Provisioning

Ehsan Moradi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
your.email@aut.ac.ir

***Abstract*—Serverless platforms keep warm containers ready for a changing arrival rate. Too few containers cause cold starts, and too many sit idle. This paper compares six forecasters of per-minute function invocations on a public Azure Functions trace: three simple baselines, seasonal Holt–Winters, a pretrained time-series model, and a small LSTM. Functions are grouped into sparse, daily, bursty, and other workloads. Each forecast covers the next 60 minutes, and the fitted models see the previous day. Error is reported beside a provisioning score that counts short and extra container-minutes, assuming one warm container serves 10 invocations a minute. On a typical function, copying the previous day has the lowest error. Forecasts that sit above the load leave almost no container short and hold many idle containers. A rule that picks the lowest-shortfall model in each class selects Chronos for sparse and daily functions. On later days that rule has a median shortfall of zero and about 1,700 extra container-minutes, while a high percentile of recent load has a median shortfall of half a container-minute and about 120 extra container-minutes.**

***Index Terms*—serverless computing, workload forecasting, cold start, provisioning**

I. Introduction

A serverless platform receives a changing number of function invocations and must decide how many warm containers to keep. A shortfall makes some requests wait for a cold start. Spare containers consume memory and money while they sit idle. A forecast of the next hour can drive that decision, but a lower forecast error does not by itself mean a better operating point. Under-predicting and over-predicting have different costs.

Forecasting is already part of production resource management. The public Azure Functions study uses a time-series forecast inside a larger keep-alive policy [1]. What is still easy to miss is that functions are not one workload. A timer with a daily shape, a rarely called HTTP function, and a bursty queue consumer can prefer different forecasters. Average error over the whole trace hides that split.

This paper is a comparative study, not a new scheduler. On 80 functions from the 2019 Azure Functions invocation trace, it asks which forecaster an operator should use for which class of function, and what that choice costs in missed invocations versus idle containers. The fitted models all see the previous day, so a daily pattern is inside their window. The contributions are a class-aware comparison, a provisioning score beside forecast error, and a selection rule fit on two days and checked on the next two.

II. Related Work

Shahrad et al. characterize the Azure Functions workload and show that most functions are invoked rarely [1]. Their hybrid policy keeps containers warm from a histogram of idle times and falls back to a time-series forecast when the histogram does not cover the idle gaps. This paper uses the same public invocation counts. It does not reimplement that keep-alive policy. The decision here is the number of containers implied by a forecast of arrivals, under a fixed service rate.

Chronos trains a probabilistic forecaster on a large collection of time series and can forecast a new series without retraining [2]. Chronos-Bolt is a faster variant that emits quantile forecasts for the whole horizon in one pass. We use the small Chronos-Bolt checkpoint with no fine-tuning, and we score its median and two upper quantiles. Upper quantiles are a direct way to buy fewer shortfalls by preparing more containers.

Learned forecasters, including recurrent networks, have been trained to pre-warm serverless containers. The question here is narrower. Under one public trace, one horizon, and one capacity assumption, with a full day of history available to every fitted model, do seasonal exponential smoothing, Chronos, or an LSTM beat simple rules that an operator can run without training?

III. Setup

The trace records invocations per minute for 14 days [1]. Class statistics use days 1–10 only. A missing day is treated as 1,440 zeros. A function is sparse when at least 90% of those minutes are zero. Otherwise it is daily when the autocorrelation at a lag of one day exceeds 0.30. A remaining function is bursty when its coefficient of variation is at least 2.05, the 75th percentile among non-sparse functions. The rest are labeled other. We draw 20 eligible functions from each class with seed 42. A function is eligible when it is invoked at least once in days 1–10 and at least once in days 11–14. The trace contains 68,394 functions in the first ten days, of which 54,502 are sparse.

xxxx-..-....-....-x/xx/$xx.00 ©2026IEEE

Each forecast predicts the next 60 minutes. A new forecast starts every 60 minutes across days 11–14, so the 96 windows cover those four days without overlap. Days 1–10 are history. Days 11–12 are used only to choose the class rule. Days 13–14 are the held-out check. Seasonal Holt–Winters, Chronos, and the LSTM each see the previous 1,440 minutes, one full day. Last-value and the trailing percentile use only the latest minutes, so a longer history would not change them.

Six point forecasts are compared. Seasonal naive repeats the same hour of the previous day when the training day-lag autocorrelation exceeds 0.30, and otherwise repeats the previous 60 minutes. Last-value repeats the latest observed minute. The trailing percentile holds the 95th percentile of the previous 10 minutes for the whole hour. ETS is additive Holt–Winters with a daily season. Its smoothing weights are chosen on days 1–10 only, separately for each function, on a grid of level, trend, and seasonal coefficients, by one-step squared error. The chosen weights then filter the series, and a forecast uses only the state from minutes before its origin. Chronos is the median forecast of `amazon/chronos-bolt-small`, applied with no training on this trace. The LSTM is one shared network with a single layer of 32 units. It is trained on windows from days 1–8, early-stopped on days 9–10, and kept at the weights with validation MAE 8.94. Each LSTM window is scaled by the mean of its own context and scaled back after prediction.

The provisioning score assumes that one warm container serves 10 invocations in a minute. For a forecast $\hat{y}_t$ of the actual count y_t,

$$S = \sum_t \max(0,\ \lceil y_t/10 \rceil - \lceil \hat{y}_t/10 \rceil)\,, \tag{1}$$

$$O = \sum_t \max(0,\ \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

S counts short container-minutes. O counts extra container-minutes. A forecast error that stays inside the same container adds nothing to either sum: eight invocations against a forecast of five still needs one container. Any forecast in $(0, 10]$ prepares one container, so a small positive value on an idle minute costs a full container-minute and scores a large sMAPE. Forecast error is the symmetric mean absolute percentage error (sMAPE). A minute where both the actual value and the forecast are zero contributes nothing. A few heavy functions dominate class sums, so every table reports the median across functions.

IV. Evaluation

Table I summarizes the four test days. Seasonal naive has the lowest overall median sMAPE, 7.1%. The gap is largest on daily functions, where its median sMAPE is 3.9%, against 40.0% for the LSTM, 60.6% for seasonal Holt–Winters, and 125.0% for Chronos. Those three models all observed the previous day. Fig. 1 shows the pattern the copy uses. On the daily example, yesterday's shape follows the load. The trailing percentile sits in steps above it. On the sparse example, arrivals are isolated spikes. The LSTM has the lowest sparse sMAPE, 0.7%, and no idle containers, because it predicts zeros. On daily functions its median container shortfall is 1,088, against 45 for seasonal naive.

Container shortfall reorders the same forecasts. Chronos has the smallest overall median, 1.0 short container-minute, because the forecast sits above the load, and it pays 3,281 extra container-minutes. The trailing percentile is next among the point forecasts that an operator might actually run, at 18.5 short container-minutes and 729 extra. Seasonal naive is short 75 container-minutes and wastes 80.5. The LSTM, which predicts low, is short 814. On daily functions both Chronos and the trailing percentile have a median shortfall of zero; Chronos wastes 3,549 container-minutes there and the percentile wastes 4,386.5. Raising Chronos from the median to the 90th percentile brings the overall median shortfall from 1.0 to 0 and raises extra container-minutes from 3,281 to 4,770. Fig. 2 shows that pair of axes. The upper quantiles lie on the zero-shortfall line.

The selection rule uses only point forecasts. On days 11–12, each class takes the forecast with the lowest median shortfall, then the lower waste, then the lower sMAPE. Sparse functions select Chronos: its median shortfall is 0, against 8.5 for seasonal naive. Daily functions select Chronos as well. The trailing percentile also has a daily shortfall of 0 on those days, and Chronos wastes less, 1,672 container-minutes against 2,113.5. Bursty and other functions select the trailing percentile. Table II applies that frozen rule to days 13–14. Across all functions the rule's median shortfall is 0 and its median waste is 1,713.5 container-minutes. Always using Chronos matches the zero shortfall and wastes 1,920. Always using the trailing percentile has a median shortfall of 0.5 and wastes 120.5. Always using seasonal naive has a median shortfall of 12 and wastes 15. On the held-out sparse functions, seasonal naive also has a shortfall of 0, with 0.5 extra container-minutes, while the rule's Chronos choice wastes 2,703.5. The shortfall rule therefore spends idle containers to buy a gap that the later days do not repeat.

The study has four limits. The trace is from 2019 and contains 80 sampled functions, not a live deployment. The capacity of 10 invocations per container-minute is an assumption, not a measured service rate. S and O are computed from the trace, not from cold starts on a platform such as OpenWhisk. Medians are the right summary here, and they are not a substitute for a cost model that weights a few very large functions the way a provider bill would.

V. Conclusion

On this trace, copying the previous day is the accurate forecast, including for functions whose shape repeats every day. Seasonal Holt–Winters, Chronos-Bolt, and a small LSTM all receive that same day and remain well above the copy on error. Measured in containers, the high forecasts are the ones with almost no shortfall, and they pay for it with idle containers. A shortfall-only rule selects Chronos for sparse and daily functions. On held-out days its median shortfall is zero and its median waste is about 1,700 container-minutes,

TABLE I
MEDIAN SMAPE, SHORT CONTAINER-MINUTES, AND EXTRA CONTAINER-MINUTES ON DAYS 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Short	13.0	45.0	718.0	150.0	75.0
	Extra	13.0	43.0	719.0	150.5	80.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Short	17.0	1150.0	763.5	99.5	410.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing percentile	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Short	15.0	0.0	143.0	0.0	18.5
	Extra	87.5	4386.5	2183.0	98.0	729.0
ETS	sMAPE (%)	112.8	60.6	116.1	15.7	66.4
	Short	3.5	21.0	250.0	0.0	15.0
	Extra	3225.5	1981.5	2939.0	1046.0	2101.5
LSTM	sMAPE (%)	0.7	40.0	43.7	41.7	40.0
	Short	19.0	1088.0	787.5	1152.0	814.0
	Extra	0.0	13.0	459.5	122.5	46.0
Chronos	sMAPE (%)	178.6	125.0	114.2	12.0	119.4
	Short	0.5	0.0	276.0	0.0	1.0
	Extra	5126.0	3549.0	2760.5	694.0	3281.0

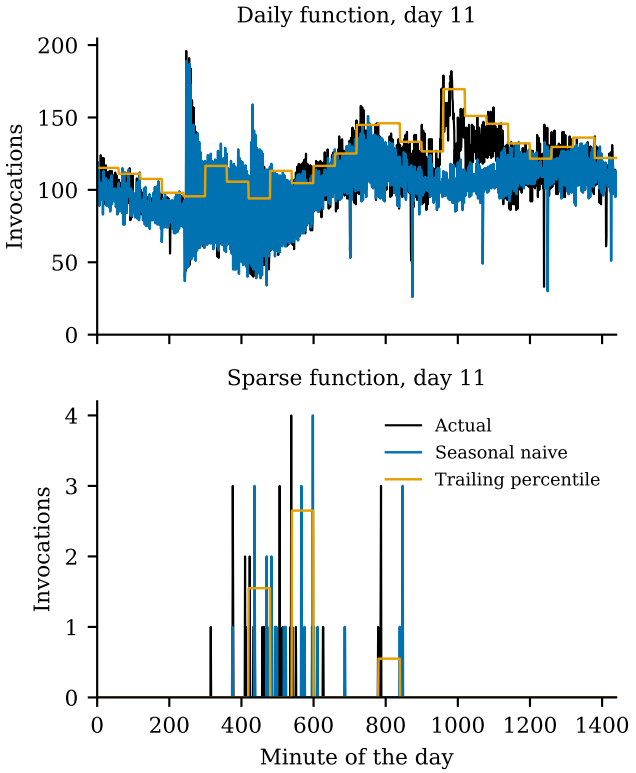


Fig. 1. Day 11 for one daily function and one sparse function. Seasonal naive follows the daily shape. The trailing percentile stays above that shape and reacts at the sparse spikes.

while the trailing percentile is short by half a container-minute and wastes about 120. A next measurement is to replace the capacity assumption with cold starts recorded on a real platform.

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218.

[2] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=gerNCVqqtR

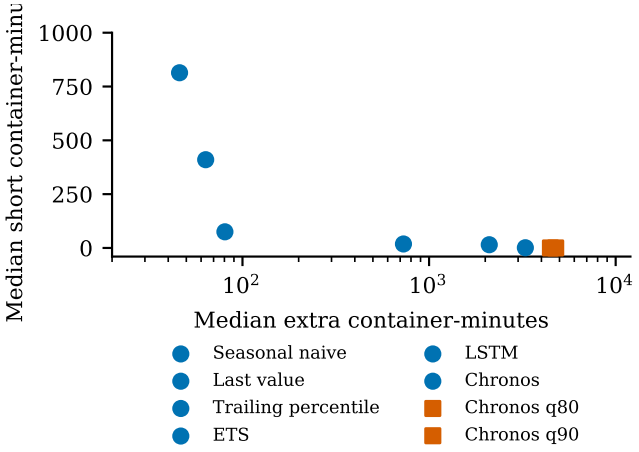


Fig. 2. Median short container-minutes against median extra container-minutes on days 11–14. The horizontal axis is logarithmic. Orange squares are Chronos forecasts at the 80th and 90th percentiles. Closer to the lower left is better.

TABLE II
HELD-OUT MEDIANS ON DAYS 13–14. THE RULE USES CHRONOS FOR SPARSE AND DAILY FUNCTIONS AND THE TRAILING PERCENTILE OTHERWISE. THE OTHER ROWS USE ONE FORECAST FOR EVERY CLASS.

Policy	Class	sMAPE (%)	Short	Extra
Selection rule	Sparse	187.7	0.0	2703.5
	Daily	149.7	0.0	1933.0
	Bursty	52.5	33.0	722.5
	Other	10.8	0.0	31.5
	All	113.6	0.0	1713.5
Seasonal naive	Sparse	0.1	0.0	0.5
	Daily	4.0	17.5	21.0
	Bursty	14.2	260.5	258.0
	Other	9.3	42.0	39.0
	All	2.7	12.0	15.0
Trailing percentile	Sparse	0.1	2.0	0.0
	Daily	134.5	0.0	2278.5
	Bursty	52.5	33.0	722.5
	Other	10.8	0.0	31.5
	All	16.2	0.5	120.5
Chronos	Sparse	187.7	0.0	2703.5
	Daily	149.7	0.0	1933.0
	Bursty	129.3	78.5	1665.5
	Other	13.5	0.0	332.5
	All	131.2	0.0	1920.0